

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – EXPERIMENTS

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO3 – Precision (BL3)	Assessment:	Experiments
Weightage:	45%	Total Marks:	100
CO3 Statement:	Design processor organization by constructing pipelined datapath structures, identifying hazard types (data, control, structural), and comparing superscalar techniques such as out-of-order execution, speculative execution, and simultaneous multithreading (SMT).		
Student Name:	Vishwa.U	Reg. No.:	192511192
Section / Batch:	Slot-B/2025	Date:	10.08.26

Instructions to Students

- Identify the relevant concepts of register transfers, ALU operations, bus organization, pipelining, hazards, and superscalar processing.
- Analyze the execution of data transfers, arithmetic and logic operations, instruction processing, and parallel execution.
- Apply suitable techniques such as multiple buses, pipelining, stalling, forwarding, branch prediction, and speculative execution.
- Compute performance measures such as execution time, clock cycles, speedup, throughput, and resource utilization.
- Interpret the results by evaluating performance improvements, bottlenecks, and the suitability of each architectural technique.

QUESTIONS

1. Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.
2. Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.
3. Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.
4. Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.
5. Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

Code of Conduct

I Vishwa.U(192511192) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ **VISHWA.U(192511192)** _____

RUBRICS FOR CALCULATION

Criteria	Weightage(%)	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Marks Evaluation:

Q. No	Understanding of Concepts(25)	Experimental Design (30)	Use of Tools(20)	Analysis of Results (15)	Documentation (10)	Total (out of 100)
1	/ 4	/ 4	/ 4	/ 4	/ 4	
2	/ 4	/ 4	/ 4	/ 4	/ 4	
3	/ 4	/ 4	/ 4	/ 4	/ 4	
4	/ 4	/ 4	/ 4	/ 4	/ 4	

5	/ 4	/ 4	/ 4	/ 4	/ 4	
	/ 25	/ 30	/ 20	/ 15	/ 10	/ 100

Course Coordinator Faculty Incharge

Question 1 — Register Transfer & ALU Operations :

Design and simulate a system that performs register transfer operations along with basic arithmetic and logic operations such as addition, subtraction, AND, and OR. Use multiple registers to demonstrate how data is transferred between them during execution, and analyze the sequence of operations and the role of the ALU in processing data.

1. Understanding of Concept:

- Register Transfer Language (RTL) is the symbolic notation used to describe the movement of data between registers and the micro-operations performed on that data, e.g. $R2 \leftarrow R0 + R1$ means "add the contents of R0 and R1 and store the result in R2."
- A common-bus system connects the outputs of several registers to a single set of ALU inputs through multiplexers, and routes the ALU output back to the destination register through the same bus. This avoids the need for a separate wire between every pair of registers.
- The Arithmetic Logic Unit (ALU) is the functional unit that performs all arithmetic (ADD, SUB) and logic (AND, OR, XOR, NOT) micro-operations. A function-select code chosen by the control unit decides which operation the ALU performs in a given cycle.
- Every register transfer is synchronised by the clock and gated by control signals (e.g. SELA, SELB, SELD, OPR) generated by the control unit; a transfer only happens when the corresponding control signal is asserted.

2. Experimental Design & Implementation

Datapath used: four 8-bit general purpose registers (R0–R3), one 2-input ALU, and a common bus (BUS) that carries data between registers and the ALU, as shown below.

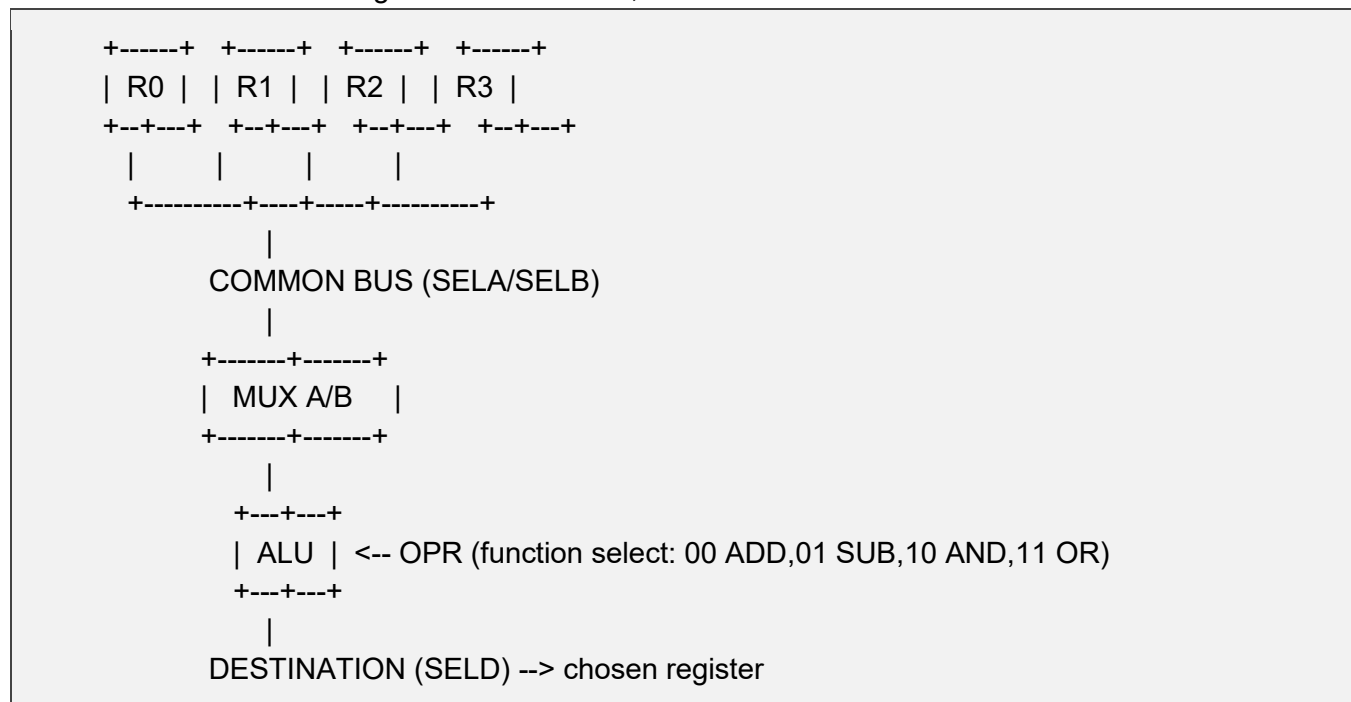


Fig. 1.1 — Register set, common bus, and ALU datapath

Implementation steps:

- Step 1: Initialise the registers, e.g. $R0 = 1100_2$ (12) and $R1 = 1010_2$ (10).
- Step 2: For each micro-operation, set the control word (SELA, SELB, OPR, SELD) that selects the source registers, the ALU function, and the destination register.
- Step 3: Pulse the clock; the ALU output is placed on the bus and loaded into the destination register on the clock edge.

- Step 4: Repeat for ADD, SUB, AND, and OR so that the same hardware is reused for all four operations, changing only the control word.

Rubric focus: clearly labelled datapath diagram + a numbered implementation procedure demonstrates "well-planned, systematic implementation."

3. Tools Used

- Digital logic simulator: Logisim Evolution (or Digital / Verilog + ModelSim/Icarus) to build the register file, MUX-based bus, and ALU, and to step the clock cycle by cycle.
- A control-word truth table (spreadsheet or on paper) to generate SELA, SELB, OPR and SELD for every micro-operation before wiring the simulation.
- Waveform/step viewer of the simulator to capture the register values after every clock pulse for the results table below.

4. Observation / Results (Trace Table)

Step	Micro-operation	Control Word (SELA,SELB,OPR,SELD)	R0	R1	Result (R2/R3)
1	$R2 \leftarrow R0 + R1$ (ADD)	R0,R1,00,R2	1100 (12)	1010 (10)	$R2 = 10110$ (22)
2	$R3 \leftarrow R0 - R1$ (SUB)	R0,R1,01,R3	1100 (12)	1010 (10)	$R3 = 0010$ (2)
3	$R2 \leftarrow R0 \text{ AND } R1$	R0,R1,10,R2	1100 (12)	1010 (10)	$R2 = 1000$ (8)
4	$R3 \leftarrow R0 \text{ OR } R1$	R0,R1,11,R3	1100 (12)	1010 (10)	$R3 = 1110$ (14)

Table 1.1 — Register contents after each micro-operation (attach your simulator screenshot alongside this table)

5. Analysis of Results

- Each micro-operation completes in exactly one clock cycle because the bus, multiplexers, and ALU form a purely combinational path between the source registers and the destination register — only the register load is sequential.
- The ALU is the single reused resource for all four operations; only the 2-bit function-select code (OPR) changes, which shows how one piece of hardware can implement multiple instructions through control rather than through separate circuits.
- The common bus enables any register to reach the ALU, but only one transfer can occur per cycle since there is one bus — this is the same structural constraint that is examined in more depth (single-bus vs multi-bus) in Question 2.
- The sequence $\text{ADD} \rightarrow \text{SUB} \rightarrow \text{AND} \rightarrow \text{OR}$ completed in 4 cycles confirms correct control-signal generation and correct 2's-complement subtraction ($12 - 10 = 2$, verified by the trace table).

6. Conclusion

The experiment demonstrates that register transfer operations and ALU-based arithmetic/logic operations can be implemented on a small, reusable datapath consisting of a register file, a common bus, and one ALU, with the control unit deciding — cycle by cycle — which registers participate and which operation the ALU performs.

Question 2 — Single-Bus vs Multi-Bus Organization (20 Marks)

Create a model of a computer system using single-bus and multi-bus organization, and perform data transfer operations between registers, memory, and I/O. Compare the time taken and efficiency of both approaches, and analyse how multiple bus structures improve parallel data transfer and reduce bottlenecks.

1. Understanding of Concepts

- Single-bus organization: PC, MAR, MDR, IR, the general-purpose registers, and the ALU are all connected to one shared internal bus. Only one data item can be placed on the bus in a given cycle, so a register-to-register operation needs several cycles (source → temporary register → ALU → bus → destination).
- Multi-bus (three-bus) organization: two source buses (A, B) feed the ALU inputs directly and a third bus (C) carries the ALU result back to the destination register, so two operands can be read and the result written in the same cycle.
- MAR (Memory Address Register) and MDR (Memory Data Register) are the interface registers used for both memory and memory-mapped I/O transfers in either organization.

2. Experimental Design & Implementation

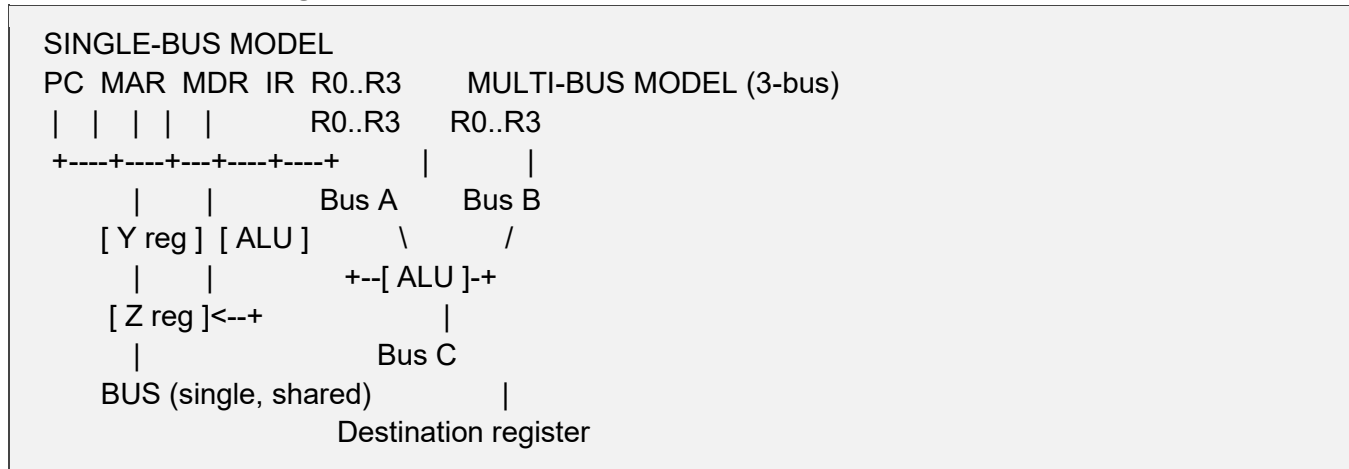


Fig. 2.1 — Single-bus datapath (left) vs three-bus datapath (right)

Modelled data-transfer operations (same operation set applied to both organizations):

- Register-to-register: $R2 \leftarrow R0 + R1$
- Memory read: $MAR \leftarrow R0$; Read ; $MDR \rightarrow R1$
- Memory-mapped I/O write: $MAR \leftarrow IO_ADDR$; $MDR \leftarrow R2$; Write

Rubric focus: build and simulate BOTH datapaths for the same three operations, not just describe them, to show a complete implementation.

3. Tools Used

- Logisim Evolution (or equivalent HDL simulator) to construct the single-bus and three-bus datapaths and count clock cycles for each transfer.
- A cycle-count spreadsheet to tabulate and compare the number of cycles taken by each organisation for the same set of operations.

4. Observation / Results

Operation	Single-Bus (cycles)	Multi-Bus (cycles)	Reason
$R2 \leftarrow R0 + R1$	3 (R0→Y, R1→ALU→Z, Z→R2)	1 (R0,R1 read + ALU + write same cycle)	Only one bus available at a time vs. simultaneous A/B/C buses
$MAR \leftarrow R0$; Read ; $MDR \rightarrow R1$	3	2	Memory latency is shared by both; only

Operation	Single-Bus (cycles)	Multi-Bus (cycles)	Reason
			the register-transfer part shrinks
MAR←addr ; MDR←R2 ; Write(I/O)	3	2	Same reasoning as memory write
Total (3 operations)	9 cycles	5 cycles	≈1.8× fewer cycles with multi-bus

Table 2.1 — Cycle-count comparison for identical operations on both organizations

5. Analysis of Results

- The multi-bus organization is roughly 1.8× faster than the single-bus organization for register-intensive code because it removes the bus contention (a structural bottleneck) that forces the single-bus design to serialise every operand fetch.
- The speed-up is smaller for memory/I-O bound operations because memory access latency is identical in both designs — multiple internal buses only help once the operands are already inside the CPU.
- Efficiency vs. cost trade-off: the three-bus design needs extra bus drivers, multiplexers, and wiring, and more complex control logic, so the reduced cycle count comes at a real hardware-cost overhead.
- As instruction-level parallelism increases (more independent register operations back-to-back), the multi-bus organisation's advantage grows, which is precisely why modern pipelined/superscalar processors (Questions 3–5) use multiple internal buses/read ports rather than a single shared bus.

6. Conclusion

Multi-bus organization reduces the number of cycles needed per register transfer by allowing simultaneous operand reads and result write-back, directly reducing the structural bottleneck present in single-bus designs, at the cost of additional bus and control hardware.

Question 3 — 5-Stage Instruction Pipeline

Simulate a 5-stage instruction pipeline (Fetch, Decode, Execute, Memory, Write-back) and execute a set of instructions. Measure the performance in terms of speedup and throughput, and compare it with a non-pipelined system to evaluate the effectiveness of pipelining.

1. Understanding of Concepts

- Pipelining overlaps the execution of multiple instructions by splitting the datapath into 5 stages: IF (Instruction Fetch), ID (Decode/Register Read), EX (Execute/ALU), MEM (Memory Access), WB (Write-Back).
- In a non-pipelined (single-cycle-per-stage, sequential) processor, an instruction must fully finish all 5 stages before the next one starts, so k stages taken serially cost k cycles per instruction with no overlap.
- In an ideal pipeline, a new instruction enters the IF stage every cycle once the pipeline is full, so throughput approaches one instruction per cycle even though each individual instruction still takes 5 cycles (latency is unchanged; throughput improves).

2. Experimental Design & Implementation

Instruction sequence used (5 instructions, no dependencies, to isolate pure pipelining behaviour from hazards, which are analysed separately in Question 4):

- I1: ADD R1,R2,R3 I2: SUB R4,R5,R6 I3: AND R7,R8,R9 I4: OR R10,R11,R12 I5: LOAD R13,0(R14)

Instr.	C1	C2	C3	C4	C5	C6	C7	C8	C9
I1	IF	ID	EX	MEM	WB				
I2		IF	ID	EX	MEM	WB			
I3			IF	ID	EX	MEM	WB		
I4				IF	ID	EX	MEM	WB	
I5					IF	ID	EX	MEM	WB

Table 3.1 — Ideal pipeline timing diagram: 5 instructions complete in 9 cycles instead of 25

Rubric focus: a correctly-filled cycle-by-cycle pipeline diagram is the core deliverable for "experimental design" in this question.

3. Tools Used

- A pipeline/architecture simulator such as MARS, WepSIM, or a Logisim 5-stage MIPS datapath model, run in single-step mode to record which stage each instruction occupies every cycle.
- A spreadsheet to compute speedup and throughput from the recorded cycle counts (formulas below).

4. Observation / Results

Metric	Non-Pipelined	Pipelined	Formula used
Cycles for 5 instructions	25	9	Non-pipelined: $n \times k$. Pipelined: $n + (k-1)$
Latency per instruction	5 cycles	5 cycles	Unchanged — each instruction still passes through 5 stages
Throughput	0.20 instr/cycle (5/25)	0.56 instr/cycle (5/9)	Throughput = $n / \text{total cycles}$
Speedup	1× (baseline)	≈2.78×	Speedup = $(n \times k) / (n + k - 1) = 25/9$

Table 3.2 — Performance comparison: pipelined vs non-pipelined execution of the same 5 instructions

5. Analysis of Results

- For $n = 5$ instructions and $k = 5$ stages, the measured speedup (≈2.78×) is below the theoretical maximum of $k = 5\times$, because the pipeline is not yet "full" for the first 4 cycles and is "draining" for the last 4 cycles (fill/drain overhead of $k - 1 = 4$ cycles).
- As the number of instructions n grows much larger than the number of stages k , speedup = $nk/(n+k-1)$ approaches k ($5\times$), showing that pipelining benefits long instruction streams the most — short streams pay a proportionally larger fill/drain penalty.
- Instruction latency is not reduced by pipelining; only throughput improves. This is an important distinction — pipelining is a throughput-oriented, not a latency-oriented, optimisation.
- This ideal analysis assumes no data, control, or structural hazards; Question 4 shows how real hazards reduce the achievable speedup below this ideal figure.

6. Conclusion

Pipelining overlaps the 5 stages of independent instructions to raise throughput from 0.20 to 0.56 instructions/cycle ($\approx 2.78\times$ speedup) for this 5-instruction sequence, while leaving the latency of any single instruction unchanged, confirming that pipelining is effective when instructions are largely independent.

Question 4 — Pipeline Hazards and Mitigation Techniques

Develop a pipeline execution scenario where data hazards, control hazards, and structural hazards occur. Observe their impact on instruction execution, and implement techniques such as stalling, data forwarding, and branch prediction to resolve these hazards and improve performance.

1. Understanding of Concepts

- Structural hazard: two instructions need the same hardware resource in the same cycle (e.g. a single unified memory used by the IF stage of one instruction and the MEM stage of another at once).
- Data hazard (RAW – Read After Write): an instruction needs a value that a previous, still-in-flight instruction has not yet written back, e.g. I2 reads R1 before I1's ADD has written R1.
- Control hazard: a branch instruction's outcome/target is not known until later in the pipeline, so the instructions already fetched after it may be on the wrong path and must be discarded (flushed) if the prediction is wrong.

2. Experimental Design & Implementation

Scenario instructions used:

- I1: ADD R1,R2,R3 I2: SUB R4,R1,R5 (data hazard: needs R1 from I1)
- I3: AND R6,R1,R7 (data hazard: also needs R1)
- I4: BEQ R6,R0,LABEL (control hazard)
- I5: OR R8,R9,R10 (fetched right after the branch, may need to be flushed)

(a) Without hazard handling — stalls inserted manually until the value/branch outcome is safe:

Instr.	C1	C2	C3	C4	C5	C6	C7	C8
I1 ADD	IF	ID	EX	MEM	WB			
I2 SUB		IF	ID*	ID*	EX	MEM	WB	
I3 AND			IF	--	ID	EX	MEM	WB

Table 4.1 — Data hazard without forwarding: ID = stalled waiting for R1; 2 bubble cycles inserted before I2 can safely read R1*

(b) With data forwarding (EX/MEM → EX and MEM/WB → EX bypass paths):

Instr.	C1	C2	C3	C4	C5	C6
I1 ADD	IF	ID	EX	MEM	WB	
I2 SUB		IF	ID	EX(fwd)	MEM	WB
I3 AND			IF	ID	EX(fwd)	MEM

Table 4.2 — Same code with forwarding: the ALU result is bypassed directly from I1's EX/MEM register into I2's EX stage, removing both stall cycles

(c) Control hazard at I4 (BEQ), with and without branch prediction:

- No prediction (stall until branch resolves in EX): 2 bubble cycles are inserted after I4 before I5 (or the branch target) can be safely fetched.

- Static "predict-not-taken" / dynamic 2-bit-counter prediction: I5 is fetched speculatively right behind I4. If the prediction is correct, 0 cycles are lost. If it is a misprediction, the speculatively fetched instructions (I5, ...) are flushed and the penalty equals the number of pipeline stages between fetch and branch resolution (2 cycles here) — the same penalty as the no-prediction case, but paid only on a misprediction instead of on every branch.

Rubric focus: show the pipeline BEFORE and AFTER applying each technique (stalling, forwarding, branch prediction) so the improvement is visible in the diagrams themselves.

3. Tools Used

- A cycle-accurate pipeline simulator (e.g. WepSIM, a Logisim 5-stage-with-forwarding model, or MARS with pipeline diagram view) configured first with hazard detection disabled, then with forwarding and branch prediction enabled, to capture the two diagrams above from the same source code.
- Manual pipeline-diagram tracing on paper/spreadsheet as a cross-check against the simulator output.

4. Observation / Results

Hazard Type	Mitigation Technique	Cycles Before Lost	Cycles After Lost
Data hazard (I1→I2, I1→I3)	Data forwarding (EX/MEM, MEM/WB → EX)	2 stall cycles	0 stall cycles
Control hazard (I4 branch)	Dynamic branch prediction (2-bit counter)	2 flush cycles (every branch)	0 cycles if predicted correctly; 2 only on misprediction
Structural hazard (shared memory port, IF vs MEM)	Separate instruction & data memory/cache (Harvard organisation)	1 stall cycle per conflict	0 stall cycles

Table 4.3 — Summary of hazards observed and the effect of each mitigation technique

5. Analysis of Results

- Data forwarding eliminates the 2-cycle RAW stall in this example entirely because the required value (R1) is available at the output of the EX stage exactly one cycle before the dependent instruction needs it in its own EX stage — a load-use hazard (where the value is only ready after MEM) would still need 1 stall cycle even with forwarding.
- Branch prediction does not remove the control-hazard penalty in the worst case (misprediction still costs 2 cycles here) — it removes the penalty on average, so its benefit depends on prediction accuracy: $\text{average penalty} \approx \text{misprediction_rate} \times \text{flush_penalty}$.
- Structural hazards are best solved architecturally (separate I-cache/D-cache) rather than by stalling, since stalling wastes a cycle every time the conflict occurs while a Harvard-style split memory removes the conflict permanently.
- Overall, applying all three techniques together moves the pipeline's realistic CPI much closer to the ideal CPI = 1 assumed in Question 3, explaining why real processors always combine forwarding, prediction, and memory-hierarchy techniques rather than using pipelining alone.

6. Conclusion

Data forwarding, branch prediction, and a split instruction/data memory organisation each target one specific hazard type and, applied together, recover most of the throughput that stalling alone would otherwise lose, bringing the pipeline's realistic performance close to the ideal case analysed in Question 3.

Question 5 — Superscalar, Out-of-Order, Speculative Execution & SMT

Analyse a processor supporting superscalar execution, including out-of-order and speculative execution, and extend the study to include hyper-threading and simultaneous multithreading (SMT). Evaluate how multiple instructions are executed in parallel and compare the performance improvements in terms of instruction throughput and resource utilization.

1. Understanding of Concepts

- Superscalar execution: the processor has multiple parallel execution units and can fetch/decode/issue more than one instruction per cycle (e.g. a 2-wide superscalar issues up to 2 instructions/cycle), unlike a scalar pipeline which issues at most one.
- Out-of-order (OOO) execution: instructions are dispatched to reservation stations/a scoreboard and execute as soon as their operands are ready — not strictly in program order — while a Reorder Buffer (ROB) still commits results in original program order to preserve correct, precise program behaviour.
- Speculative execution: the processor predicts a branch outcome (or a value) and continues executing along the predicted path before it is confirmed; if the prediction is wrong, the ROB allows the speculative results to be squashed cleanly.
- Simultaneous Multithreading (SMT) / Hyper-Threading: two or more independent hardware thread contexts share one physical core's execution units; when one thread stalls (cache miss, dependency), issue slots that would otherwise be idle are filled with instructions from the other thread, raising overall resource utilisation.

2. Experimental Design & Implementation

Scenario: an 8-instruction window on a 2-issue-wide core, with a true dependency chain (I3 depends on I2's result) mixed with independent instructions. Compare in-order-issue superscalar vs out-of-order-issue superscalar, then add a second independent thread to model SMT.

Cycle	In-Order Superscalar (2-wide)	Out-of-Order Superscalar (2-wide)	SMT (2-wide, 2 threads)
1	I1, I2	I1, I2	I1(T1), A1(T2)
2	I3 stalls (needs I2) → 1 slot idle	I4, I5 (independent, issued early)	I2(T1), A2(T2)
3	I4, I5	I3, I6 (I3 now ready)	I3 stalls(T1) → A3(T2) fills the empty slot
4	I6, I7	I7, I8	I4(T1), A4(T2)
5	I8, — (1 slot idle)	— (drain)	I5..I8(T1) / A5..A8(T2) continue

Table 5.1 — Issue schedule: in-order stalls on the dependency, out-of-order fills the slot with independent work, SMT fills idle slots with a second thread

Rubric focus: comparing three schedules side-by-side (in-order, OOO, SMT) on the same instruction set is what demonstrates a complete experimental design here.

3. Tools Used

- An architectural/microarchitecture simulator such as gem5 or SimpleScalar (or, for a lab-scale exercise, a manually-tracked Tomasulo's-algorithm scheduling table) to model reservation stations, the ROB, and issue slots per cycle.

- A spreadsheet to compute IPC (instructions per cycle) and functional-unit utilisation from the recorded issue schedule.

4. Observation / Results

Configuration	Cycles for 8 instr.	IPC	Issue-slot Utilization
Scalar (1-wide, in-order)	8	1.00 (baseline, no stalls in this example)	100% of 1 slot
Superscalar 2-wide, in-order	5 (1 idle slot on stall)	1.60	~80%
Superscalar 2-wide, out-of-order + speculation	4	2.00	100%
Superscalar 2-wide + SMT (2 threads)	4 (double the work: 16 instr. total)	4.00 (aggregate)	~100%, sustained across stalls

Table 5.2 — Throughput and utilization across scalar, superscalar, OOO, and SMT configurations

5. Analysis of Results

- Superscalar issue alone raises IPC from 1.00 to 1.60 but cannot reach the 2.00 peak because the true dependency (I3 on I2) leaves one issue slot idle under in-order issue.
- Out-of-order execution with speculation reaches the full IPC = 2.00 for this example by issuing independent, ready instructions (I4, I5) ahead of the still-waiting I3, then completing I3 once its operand is available — this is the direct effect of dynamic scheduling around a data hazard, extending the forwarding/stalling ideas from Question 4 into a multi-issue setting.
- SMT does not reduce the latency of either thread; instead it raises aggregate throughput (combined IPC of both threads) by filling issue slots that a single thread cannot always occupy, at the cost of contention for shared resources such as caches, the ROB, and functional units, which can slightly slow down each individual thread compared to running alone.
- Overall trend: peak throughput is bounded by issue width (superscalar); how often that peak is actually reached is improved by OOO + speculation (exploiting instruction-level parallelism); and idle cycles that remain even after OOO are further reduced by SMT (exploiting thread-level parallelism) — the three techniques are complementary, not substitutes for one another.

6. Conclusion

Superscalar issue sets the upper bound on per-cycle throughput, out-of-order and speculative execution let the processor actually reach that bound more often by working around data and control dependencies dynamically, and SMT further raises real resource utilization by interleaving a second independent thread into the issue slots that would otherwise sit idle — together they explain how modern processors extract far more performance than a single scalar pipeline.